Search API Docs

dio

pub v5.11.0 dev v5.11.0

Language: English | 简体中文

A powerful HTTP networking package for Dart/Flutter, supports Global configuration, Interceptors, FormData, Request cancellation, File uploading/downloading, Timeout, Custom adapters, Transformers, etc.

> Don't forget to add #dio topic to your published dio related packages! See more: https://dart.dev/tools/pub/pubspec#topics

▶ Table of content

Get started

Install

Add the `dio` package to your pubspec dependencies.

Before you upgrade: Breaking changes might happen in major and minor versions of packages.
See the Migration Guide for the complete breaking changes list.

Super simple to use

```dart
import 'package:dio/dio.dart';

final dio = Dio();

void getHttp() async {
  final response = await dio.get('https://dart.dev');
  print(response);
}
```

Awesome dio

🎉 A curated list of awesome things related to dio.

Plugins

Plugins

Welcome to submit third-party plugins and related libraries in here.

Examples

Performing a `GET` request

```dart
import 'package:dio/dio.dart';

final dio = Dio();

void request() async {
  Response response;
  response = await dio.get('/test?id=12&name=dio');
  print(response.data.toString());
  // The below request is the same as above.
  response = await dio.get(
    '/test',
    queryParameters: {'id': 12, 'name': 'dio'},
  );
  print(response.data.toString());
}
```

Performing a `POST` request

```dart
response = await dio.post('/test', data: {'id': 12, 'name': 'dio'});
```

Performing a `QUERY` request

The `QUERY` method (RFC 10008) is safe and idempotent like `GET`, but allows a request body, which is useful for complex queries that do not fit in the URL.

```dart
response = await dio.query('/search', data: {'filter': {'name': 'dio'}});
```

Performing multiple concurrent requests

```
List<Response> responses = await Future.wait([dio.post('/info'), dio.get('/tok
```

Downloading a file

```
response = await dio.download(
  'https://pub.dev/',
  (await getTemporaryDirectory()).path + 'pub.html',
);
```

On Web, the second argument is used as the browser's suggested filename instead of a local filesystem path. The browser chooses the actual saved location, the response is loaded into memory before the download is triggered, and CORS still applies. `FileAccessMode.append` is not supported, `deleteOnError` has no local file to delete, and custom `lengthHeader` values are not used for Web progress totals.

Get response stream

```
final rs = await dio.get(
  url,
  options: Options(responseType: ResponseType.stream), // Set the response type
);
print(rs.data.stream); // Response stream.
```

Get response with bytes

```
final rs = await Dio().get<List<int>>(
  url,
  options: Options(responseType: ResponseType.bytes), // Set the response type
);
print(rs.data); // Type: List<int>.
```

Sending a `FormData`

```dart
final formData = FormData.fromMap({
  'name': 'dio',
  'date': DateTime.now().toIso8601String(),
});
final response = await dio.post('/info', data: formData);
```

Uploading multiple files to server by FormData

```dart
final formData = FormData.fromMap({
  'name': 'dio',
  'date': DateTime.now().toIso8601String(),
  'file': await MultipartFile.fromFile('./text.txt', filename: 'upload.txt'),
  'files': [
    await MultipartFile.fromFile('./text1.txt', filename: 'text1.txt'),
    await MultipartFile.fromFile('./text2.txt', filename: 'text2.txt'),
  ]
});
final response = await dio.post('/info', data: formData);
```

Listening the uploading progress

```dart
final response = await dio.post(
  'https://www.dtworkroom.com/doris/1/2.0.0/test',
  data: {'aa': 'bb' * 22},
  onSendProgress: (int sent, int total) {
    print('$sent $total');
  },
);
```

Post binary data with Stream

```dart
// Binary data
final postData = <int>[0, 1, 2];
await dio.post(
  url,
  data: Stream.fromIterable(postData.map((e) => [e])), // Creates a Stream<List
  options: Options(
    headers: {
```

```
        Headers.contentLengthHeader: postData.length, // Set the content-length.
      },
    ),
  );
```

Note: `content-length` must be set if you want to subscribe to the sending progress.

See all examples code here.

Dio APIs

Creating an instance and set default configs

> It is recommended to use a singleton of `Dio` in projects, which can manage configurations like headers, base urls, and timeouts consistently. Here is an example that use a singleton in Flutter.

You can create instance of Dio with an optional `BaseOptions` object:

```dart
final dio = Dio(); // With default `Options`.

void configureDio() {
  // Set default configs
  dio.options.baseUrl = 'https://api.pub.dev';
  dio.options.connectTimeout = Duration(seconds: 5);
  dio.options.receiveTimeout = Duration(seconds: 3);

  // Or create `Dio` with a `BaseOptions` instance.
  final options = BaseOptions(
    baseUrl: 'https://api.pub.dev',
    connectTimeout: Duration(seconds: 5),
    receiveTimeout: Duration(seconds: 3),
  );
  final anotherDio = Dio(options);

  // Or clone the existing `Dio` instance with all fields.
  final clonedDio = dio.clone();
}
```

The core API in Dio instance is:

```
Future<Response<T>> request<T>(
  String path, {
  Object? data,
  Map<String, dynamic>? queryParameters,
  CancelToken? cancelToken,
  Options? options,
  ProgressCallback? onSendProgress,
  ProgressCallback? onReceiveProgress,
});
```

```
final response = await dio.request(
  '/test',
  data: {'id': 12, 'name': 'dio'},
  options: Options(method: 'GET'),
);
```

Request Options

There are two request options concepts in the Dio library: `BaseOptions` and `Options`. The `BaseOptions` include a set of base settings for each `Dio()`, and the `Options` describes the configuration for a single request. These options will be merged when making requests. The `Options` declaration is as follows:

```
/// The HTTP request method.
String method;

/// Timeout when sending data.
///
/// Throws the [DioException] with
/// [DioExceptionType.sendTimeout] type when timed out.
///
/// `null` or `Duration.zero` means no timeout limit.
Duration? sendTimeout;

/// Timeout when receiving data.
///
/// The timeout represents:
///  - a timeout before the connection is established
///    and the first received response bytes.
///  - the duration during data transfer of each byte event,
///    rather than the total duration of the receiving.
```

```dart
  ///
  /// Throws the [DioException] with
  /// [DioExceptionType.receiveTimeout] type when timed out.
  ///
  /// `null` or `Duration.zero` means no timeout limit.
  Duration? receiveTimeout;

  /// Timeout when transforming response data.
  ///
  /// Throws the [DioException] with
  /// [DioExceptionType.transformTimeout] type when timed out.
  /// On web, timeout handling is best-effort because synchronous JavaScript
  /// work cannot be preempted.
  ///
  /// `null` or `Duration.zero` means no timeout limit.
  Duration? transformTimeout;

  /// Custom field that you can retrieve it later in [Interceptor],
  /// [Transformer] and the [Response.requestOptions] object.
  Map<String, dynamic>? extra;

  /// HTTP request headers.
  ///
  /// The keys of the header are case-insensitive,
  /// e.g.: `content-type` and `Content-Type` will be treated as the same key.
  Map<String, dynamic>? headers;

  /// Whether the case of header keys should be preserved.
  ///
  /// Defaults to false.
  ///
  /// This option WILL NOT take effect on these circumstances:
  /// - XHR ([HttpRequest]) does not support handling this explicitly.
  /// - The HTTP/2 standard only supports lowercase header keys.
  bool? preserveHeaderCase;

  /// The type of data that [Dio] handles with options.
  ///
  /// The default value is [ResponseType.json].
  /// [Dio] will parse response string to JSON object automatically
  /// when the content-type of response is [Headers.jsonContentType].
  ///
  /// See also:
  ///  - `plain` if you want to receive the data as `String`.
  ///  - `bytes` if you want to receive the data as the complete bytes.
  ///  - `stream` if you want to receive the data as streamed binary bytes.
  ResponseType? responseType;

  /// The request content-type.
```

```dart
  ///
  /// The default `content-type` for requests will be implied by the
  /// [ImplyContentTypeInterceptor] according to the type of the request payload.
  /// The interceptor can be removed by
  /// [Interceptors.removeImplyContentTypeInterceptor].
  String? contentType;

  /// Defines whether the request is considered to be successful
  /// with the given status code.
  /// The request will be treated as succeed if the callback returns true.
  ValidateStatus? validateStatus;

  /// Whether to retrieve the data if status code indicates a failed request.
  ///
  /// Defaults to true.
  bool? receiveDataWhenStatusError;

  /// See [HttpClientRequest.followRedirects].
  ///
  /// Defaults to true.
  bool? followRedirects;

  /// The maximum number of redirects when [followRedirects] is `true`.
  /// [RedirectException] will be thrown if redirects exceeded the limit.
  ///
  /// Defaults to 5.
  int? maxRedirects;

  /// See [HttpClientRequest.persistentConnection].
  ///
  /// Defaults to true.
  bool? persistentConnection;

  /// The default request encoder is [Utf8Encoder], you can set custom
  /// encoder by this option.
  RequestEncoder? requestEncoder;

  /// The default response decoder is [Utf8Decoder], you can set custom
  /// decoder by this option, it will be used in [Transformer].
  ResponseDecoder? responseDecoder;

  /// Indicates the format of collection data in request query parameters and
  /// `x-www-url-encoded` body data.
  ///
  /// Defaults to [ListFormat.multi].
  ListFormat? listFormat;
```

There is a complete example here.

Response

The response for a request contains the following information.

```dart
/// Response body. may have been transformed, please refer to [ResponseType].
T? data;

/// The corresponding request info.
RequestOptions requestOptions;

/// HTTP status code.
int? statusCode;

/// Returns the reason phrase associated with the status code.
/// The reason phrase must be set before the body is written
/// to. Setting the reason phrase after writing to the body.
String? statusMessage;

/// Whether this response is a redirect.
/// ** Attention **: Whether this field is available depends on whether the
/// implementation of the adapter supports it or not.
bool isRedirect;

/// The series of redirects this connection has been through. The list will be
/// empty if no redirects were followed. [redirects] will be updated both
/// in the case of an automatic and a manual redirect.
///
/// ** Attention **: Whether this field is available depends on whether the
/// implementation of the adapter supports it or not.
List<RedirectRecord> redirects;

/// Custom fields that only for the [Response].
Map<String, dynamic> extra;

/// Response headers.
Headers headers;
```

When request is succeed, you will receive the response as follows:

```dart
final response = await dio.get('https://pub.dev');
print(response.data);
print(response.headers);
```

```
print(response.requestOptions);
print(response.statusCode);
```

Be aware, the `Response.extra` is different from `RequestOptions.extra`, they are not related to each other.

Interceptors

For each dio instance, we can add one or more interceptors, by which we can intercept requests, responses, and errors before they are handled by `then` or `catchError`.

```
dio.interceptors.add(
  InterceptorsWrapper(
    onRequest: (RequestOptions options, RequestInterceptorHandler handler) {
      // Do something before request is sent.
      // If you want to resolve the request with custom data,
      // you can resolve a `Response` using `handler.resolve(response)`.
      // If you want to reject the request with a error message,
      // you can reject with a `DioException` using `handler.reject(dioError)`.
      return handler.next(options);
    },
    onResponse: (Response response, ResponseInterceptorHandler handler) {
      // Do something with response data.
      // If you want to reject the request with a error message,
      // you can reject a `DioException` object using `handler.reject(dioError)`
      return handler.next(response);
    },
    onError: (DioException error, ErrorInterceptorHandler handler) {
      // Do something with response error.
      // If you want to resolve the request with some custom data,
      // you can resolve a `Response` object using `handler.resolve(response)`.
      return handler.next(error);
    },
  ),
);
```

Simple interceptor example:

```
import 'package:dio/dio.dart';
class CustomInterceptors extends Interceptor {
  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler handler) {
```

```
    print('REQUEST[${options.method}] => PATH: ${options.path}');
    super.onRequest(options, handler);
  }

  @override
  void onResponse(Response response, ResponseInterceptorHandler handler) {
    print('RESPONSE[${response.statusCode}] => PATH: ${response.requestOptions
    super.onResponse(response, handler);
  }

  @override
  Future onError(DioException err, ErrorInterceptorHandler handler) async {
    print('ERROR[${err.response?.statusCode}] => PATH: ${err.requestOptions.pa
    super.onError(err, handler);
  }
}
```

Resolve and reject the request

In all interceptors, you can interfere with their execution flow. If you want to resolve the request/response with some custom data, you can call `handler.resolve(Response)`. If you want to reject the request/response with a error message, you can call `handler.reject(dioError)`.

```
dio.interceptors.add(
  InterceptorsWrapper(
    onRequest: (options, handler) {
      return handler.resolve(
        Response(requestOptions: options, data: 'fake data'),
      );
    },
  ),
);
final response = await dio.get('/test');
print(response.data); // 'fake data'
```

QueuedInterceptor

`Interceptor` can be executed concurrently, that is, all the requests enter the interceptor at once, rather than executing sequentially. However, in some cases we expect that requests enter the interceptor sequentially like #590. Therefore, we need to provide a mechanism for sequential access (step by step) to interceptors and `QueuedInterceptor` can solve this problem.

Example

Because of security reasons, we need all the requests to set up a `csrfToken` in the header, if `csrfToken` does not exist, we need to request a csrfToken first, and then perform the network request, because the request csrfToken progress is asynchronous, so we need to execute this async request in request interceptor.

For the complete code see here.

LogInterceptor

You can apply the `LogInterceptor` to log requests and responses automatically.

Note: `LogInterceptor` should always be the last interceptor added, otherwise modifications by following interceptors will not be logged.

Dart

```
dio.interceptors.add(LogInterceptor(responseBody: false)); // Do not output res
```

Note: When using the default `logPrint` function, logs will only be printed in DEBUG mode (when the assertion is enabled).

Alternatively `dart:developer` 's log can also be used to log messages (available in Flutter too).

Flutter

When using Flutter, Flutters own `debugPrint` function should be used.

This ensures, that debug messages are also available via `flutter logs` .

Note: `debugPrint` **does not mean print logs under the DEBUG mode**, it's a throttled function which helps to print full logs without truncation. Do not use it under any production environment unless you're intended to.

```
dio.interceptors.add(
  LogInterceptor(
    logPrint: (o) => debugPrint(o.toString()),
  ),
);
```

Custom Interceptor

You can customize interceptor by extending the `Interceptor/QueuedInterceptor` class. There is an example that implementing a simple cache policy: custom cache interceptor.

Handling Errors

When an error occurs, Dio will wrap the `Error/Exception` to a `DioException`:

```dart
try {
  // 404
  await dio.get('https://api.pub.dev/not-exist');
} on DioException catch (e) {
  // The request was made and the server responded with a status code
  // that falls out of the range of 2xx and is also not 304.
  if (e.response != null) {
    print(e.response.data)
    print(e.response.headers)
    print(e.response.requestOptions)
  } else {
    // Something happened in setting up or sending the request that triggered an
    print(e.requestOptions)
    print(e.message)
  }
}
```

DioException

```dart
/// The request info for the request that throws exception.
RequestOptions requestOptions;

/// Response info, it may be `null` if the request can't reach to the
/// HTTP server, for example, occurring a DNS error, network is not available.
Response? response;

/// The type of the current [DioException].
DioExceptionType type;

/// The original error/exception object;
/// It's usually not null when `type` is [DioExceptionType.unknown].
Object? error;

/// The stacktrace of the original error/exception object;
/// It's usually not null when `type` is [DioExceptionType.unknown].
StackTrace? stackTrace;
```

```
/// The error message that throws a [DioException].
String? message;
```

DioExceptionType

See the source code.

Using application/x-www-form-urlencoded format

By default, Dio serializes request data (except `String` type) to `JSON`. To send data in the `application/x-www-form-urlencoded` format instead:

```
// Instance level
dio.options.contentType = Headers.formUrlEncodedContentType;
// or only works once
dio.post(
  '/info',
  data: {'id': 5},
  options: Options(contentType: Headers.formUrlEncodedContentType),
);
```

Sending FormData

You can also send `FormData` with Dio, which will send data in the `multipart/form-data`, and it supports uploading files.

```
final formData = FormData.fromMap({
  'name': 'dio',
  'date': DateTime.now().toIso8601String(),
  'file': await MultipartFile.fromFile('./text.txt', filename: 'upload.txt'),
});
final response = await dio.post('/info', data: formData);
```

You can also specify your desired boundary name which will be used to construct boundaries of every `FormData` with additional prefix and suffix.

```
final formDataWithBoundaryName = FormData(
  boundaryName: 'my-boundary-name',
```

```
);
```

> `FormData` is supported with the POST method typically.

There is a complete example here.

Multiple files upload

There are two ways to add multiple files to `FormData`, the only difference is that upload keys are different for array types。

```
final formData = FormData.fromMap({
  'files': [
    MultipartFile.fromFileSync('path/to/upload1.txt', filename: 'upload1.txt')
    MultipartFile.fromFileSync('path/to/upload2.txt', filename: 'upload2.txt')
  ],
});
```

The upload key eventually becomes `files[]`. This is because many back-end services add a middle bracket to key when they get an array of files. **If you don't want a list literal**, you should create FormData as follows (Don't use `FormData.fromMap`):

```
final formData = FormData();
formData.files.addAll([
  MapEntry(
   'files',
    MultipartFile.fromFileSync('./example/upload.txt',filename: 'upload.txt'),
  ),
  MapEntry(
    'files',
    MultipartFile.fromFileSync('./example/upload.txt',filename: 'upload.txt'),
  ),
]);
```

Reuse `FormData`s and `MultipartFile`s

You should make a new `FormData` or `MultipartFile` every time in repeated requests. A typical wrong behavior is setting the `FormData` as a variable and using it in every request. It can

be easy for the *Cannot finalize* exceptions to occur. To avoid that, write your requests like the below code:

```
Future<void> _repeatedlyRequest() async {
  Future<FormData> createFormData() async {
    return FormData.fromMap({
      'name': 'dio',
      'date': DateTime.now().toIso8601String(),
      'file': await MultipartFile.fromFile('./text.txt',filename: 'upload.txt'
    });
  }

  await dio.post('some-url', data: await createFormData());
}
```

Transformer

`Transformer` allows changes to the request/response data before it is sent/received to/from the server. Dio has already implemented a `BackgroundTransformer` as default, which calls `jsonDecode` in an isolate if the response is larger than 50 KB. If you want to customize the transformation of request/response data, you can provide a `Transformer` by your self, and replace the `BackgroundTransformer` by setting the `dio.transformer`.

> `Transformer.transformRequest` only takes effect when request with `PUT` / `POST` / `PATCH`, they're methods that can contain the request body. `Transformer.transformResponse` however, can be applied to all types of responses.

Transformer example

There is an example for customizing Transformer.

HttpClientAdapter

`HttpClientAdapter` is a bridge between `Dio` and `HttpClient`.

`Dio` implements standard and friendly APIs for developer. `HttpClient` is the real object that makes Http requests.

We can use any `HttpClient` not just `dart:io:HttpClient` to make HTTP requests. And all we need is providing a `HttpClientAdapter`. The default `HttpClientAdapter` for Dio is

`IOHttpClientAdapter` on native platforms, and `BrowserHttpClientAdapter` on the Web platform. They can be initiated by calling the `HttpClientAdapter()`.

```
dio.httpClientAdapter = HttpClientAdapter();
```

If you want to use platform adapters explicitly:

- For the Web platform:

```
import 'package:dio/browser.dart';
// ...
dio.httpClientAdapter = BrowserHttpClientAdapter();
```

- For native platforms:

```
import 'package:dio/io.dart';
// ...
dio.httpClientAdapter = IOHttpClientAdapter();
```

Here is a simple example to custom adapter.

Using proxy

`IOHttpClientAdapter` provide a callback to set proxy to `dart:io:HttpClient`, for example:

```
import 'package:dio/io.dart';

void initAdapter() {
  dio.httpClientAdapter = IOHttpClientAdapter(
    createHttpClient: () {
      final client = HttpClient();
      // Config the client.
      client.findProxy = (uri) {
        // Forward all request to proxy "localhost:8888".
        // Be aware, the proxy should went through you running device,
        // not the host platform.
        return 'PROXY localhost:8888';
      };
      // You can also create a new HttpClient for Dio instead of returning,
      // but a client must being returned here.
```

```
        return client;
      },
    );
  }
```

There is a complete example here.

Web does not support to set proxy.

HTTPS certificate verification

HTTPS certificate verification (or public key pinning) refers to the process of ensuring that the certificates protecting the TLS connection to the server are the ones you expect them to be. The intention is to reduce the chance of a man-in-the-middle attack. The theory is covered by OWASP.

Server Response Certificate

Unlike other methods, this one works with the certificate of the server itself.

```
void initAdapter() {
  const String fingerprint = 'ee5ce1dfa7a53657c545c62b65802e4272878dabd65c0aad
  dio.httpClientAdapter = IOHttpClientAdapter(
    createHttpClient: () {
      // Don't trust any certificate just because their root cert is trusted.
      final HttpClient client = HttpClient(context: SecurityContext(withTruste
      // You can test the intermediate / root cert here. We just ignore it.
      client.badCertificateCallback = (cert, host, port) => true;
      return client;
    },
    validateCertificate: (cert, host, port) {
      // Check that the cert fingerprint matches the one we expect.
      // We definitely require _some_ certificate.
      if (cert == null) {
        return false;
      }
      // Validate it any way you want. Here we only check that
      // the fingerprint matches the OpenSSL SHA256.
      return fingerprint == sha256.convert(cert.der).toString();
    },
  );
}
```

You can use openssl to read the SHA256 value of a certificate:

```
openssl s_client -servername pinning-test.badssl.com -connect pinning-test.bad
  | openssl x509 -noout -fingerprint -sha256

# SHA256 Fingerprint=EE:5C:E1:DF:A7:A5:36:57:C5:45:C6:2B:65:80:2E:42:72:87:8D:AB
# (remove the formatting, keep only lower case hex characters to match the `sha2
```

Certificate Authority Verification

These methods work well when your server has a self-signed certificate, but they don't work for certificates issued by a 3rd party like AWS or Let's Encrypt.

There are two ways to verify the root of the https certificate chain provided by the server. Suppose the certificate format is PEM, the code like:

```
void initAdapter() {
  String PEM = 'XXXXX'; // root certificate content
  dio.httpClientAdapter = IOHttpClientAdapter(
    createHttpClient: () {
      final client = HttpClient();
      client.badCertificateCallback = (X509Certificate cert, String host, int
        return cert.pem == PEM; // Verify the certificate.
      };
      return client;
    },
  );
}
```

Another way is creating a `SecurityContext` when create the `HttpClient`:

```
void initAdapter() {
  String PEM = 'XXXXX'; // root certificate content
  dio.httpClientAdapter = IOHttpClientAdapter(
    onHttpClientCreate: (_) {
      final SecurityContext sc = SecurityContext();
      sc.setTrustedCertificates(File(pathToTheCertificate));
      final HttpClient client = HttpClient(context: sc);
      return client;
    },
```

```
    );
  }
```

In this way, the format of `setTrustedCertificates()` must be PEM or PKCS12. PKCS12 requires password to use, which will expose the password in the code, so it's not recommended to use in common cases.

HTTP/2 support

dio_http2_adapter is a Dio `HttpClientAdapter` which supports HTTP/2.

Cancellation

You can cancel a request using a `CancelToken`. One token can be shared with multiple requests. When a token's `cancel()` is invoked, all requests with this token will be cancelled.

```dart
final cancelToken = CancelToken();
dio.get(url, cancelToken: cancelToken).catchError((DioException error) {
  if (CancelToken.isCancel(error)) {
    print('Request canceled: ${error.message}');
  } else {
    // handle error.
  }
});
// Cancel the requests with "cancelled" message.
token.cancel('cancelled');
```

There is a complete example here.

Extends Dio class

`Dio` is an abstract class with factory constructor, so we don't extend `Dio` class direct. We can extend `DioForNative` or `DioForBrowser` instead, for example:

```dart
import 'package:dio/dio.dart';
import 'package:dio/io.dart';
// If in browser, import 'package:dio/browser.dart'.

class Http extends DioForNative {
  Http([BaseOptions options]) : super(options) {
    // do something
```

```
    }
  }
```

We can also implement a custom `Dio` client:

```
class MyDio with DioMixin implements Dio {
  // ...
}
```

Cross-Origin Resource Sharing on Web (CORS)

If a request is not a simple request, the Web browser will send a CORS preflight request that checks to see if the CORS protocol is understood and a server is aware using specific methods and headers.

You can modify your requests to match the definition of simple request, or add a CORS middleware for your service to handle CORS requests.

Libraries

browser

io

Migration Guide

Migration Guide

dio **MIGRATION GUIDE** **PLUGINS**
A powerful HTTP client for Dart and Flutter, which supports global settings, Interceptors, FormData, aborting and canceling a request, files uploading and downloading, requests timeout, custom adapters, etc.

Plugins

Repository Version Description dio_cookie_manager v3.5.0 A cookie manager for Dio dio_http2_adapter v2.8.0 A Dio HttpClientAdapter which support Http/2.0 native_dio_adapter v1.8.0 An adapter for Dio which makes use of cupertino_http and cronet_http to delegate HTTP

requests to the native platform. dio_smart_retry v7.0.1 Flexible retry library for Dio http_certificate_pinning v3.0.2 Https Certificate pinning for Flutter dio_intercept_to_curl v0.2.0 A Flutter curl-command generator for Dio. dio_cache_interceptor v4.0.7 Dio HTTP cache interceptor with multiple stores respecting HTTP directives (or not) dio_http_cache v0.3.0 A simple cache library for Dio like Rxcache in Android pretty_dio_logger v1.4.0 Pretty Dio logger is a Dio interceptor that logs network calls in a pretty, easy to read format. dio_image_provider v0.0.2 An image provider which makes use of package:dio to instead of dart:io flutter_ume_kit_dio v1.0.1+2 A debug kit of dio on flutter_ume sentry_dio v9.27.0 An integration which adds support for performance tracing for the Dio package. talker_dio_logger v5.1.20 Colorful and customizable dio logger with a lightweight design and talker additional functionality

dio **MIGRATION GUIDE** **PLUGINS**

A powerful HTTP client for Dart and Flutter, which supports global settings, Interceptors, FormData, aborting and canceling a request, files uploading and downloading, requests timeout, custom adapters, etc.

dio 5.11.0